

Лабораторна робота 2: Контейнеризація системи

Завдання 1. Створення Dockerfile

1.1. Dockerfile для бекенду (FastAPI)

Створено файл backend/Dockerfile з такими властивостями: - Базовий образ: python:3.12-slim (офіційний мінімальний Python-образ). - Встановлюються лише залежності з requirements.txt. - Кешування шарів: COPY requirements.txt і RUN pip install ... виконуються до копіювання коду. - Сервер запускається при старті контейнера: uvicorn main:app --host 0.0.0.0 --port 8080.

Додатково створено backend/.dockerignore.

1.2. Dockerfile для фронтенду (multi-stage)

Створено файл frontend/Dockerfile: - Stage 1 (build): node:20-alpine, встановлення залежностей (npm ci), збірка (npm run build). - Stage 2 (production): nginx:1.27-alpine, копіювання статичних файлів з build stage.

Додатково створено frontend/.dockerignore.

1.3. nginx.conf для фронтенду

Створено frontend/nginx.conf: - Роздача статичних файлів з /usr/share/nginx/html. - Проксіювання /api/ на http://backend:8080. - Передача заголовків Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto. - SPA fallback: try_files \$uri \$uri/ /index.html;.

1.4. Команди збірки та перевірки

Виконано команди з умови:

```
docker build -t todo-backend ./backend
docker images | grep todo-backend
docker build -t todo-frontend ./frontend
docker build -t todo-frontend ./frontend
docker run --rm todo-frontend nginx -t
docker images | grep todo
```

Ключові результати: - todo-backend успішно зібрано. - todo-frontend успішно зібрано. - nginx -t: syntax is ok, test is

successful. - Розміри образів: - todo-backend: 165MB - todo-frontend: 48.4MB

1.5. Відповіді на запитання

1. Чим відрізняється Docker-образ від контейнера?
 - Образ: незмінний шаблон (read-only) з файловою системою та метаданими.
 - Контейнер: запущений екземпляр образу з власним runtime-станом.
2. Що відбудеться з даними в базі, якщо видалити контейнер PostgreSQL без volume? З volume?
 - Без volume: дані втрачаються разом із контейнером.
 - З volume: дані зберігаються у Docker volume і доступні після повторного створення контейнера.
3. Чим відрізняється EXPOSE в Dockerfile від -p при docker run?
 - EXPOSE: декларація порту всередині образу (документація/метадані).
 - -p: реальна публікація порту контейнера на хост.
4. Який образ найбільший і чому?
 - todo-backend (165MB) більший за todo-frontend (48.4MB), бо містить Python runtime та встановлені Python-залежності (включно з psycopg2-binary, sqlalchemy тощо).
5. Як multi-stage build вплинув на розмір образу фронтенду?
 - Значно зменшив production-образ: у фінальний образ не потрапили Node.js, node_modules та build-інструменти; лишились тільки статичні файли + nginx.

1.6. Посилання на створені файли в репозиторії

- backend/Dockerfile
- backend/.dockerignore
- frontend/Dockerfile
- frontend/nginx.conf
- frontend/.dockerignore

Завдання 2. Запуск системи

2.1. Запуск контейнерів вручну через docker run

Створено мережу:

```
docker network create todo-network
```

Запуск у практично доцільному порядку: 1. DB 2. Backend 3. Frontend

Повні команди запуску:

```
docker volume create todo-db-data
```

```
docker run -d --name db --network todo-network \
-e POSTGRES_USER=todouser \
-e POSTGRES_PASSWORD=todopass \
-e POSTGRES_DB=tododb \
-v todo-db-data:/var/lib/postgresql/data \
postgres:16
```

```
docker run -d --name backend --network todo-network \
-e DATABASE_URL=postgresql://todouser:todopass@db:5432/tododb \
-e APP_HOST=0.0.0.0 \
-e APP_PORT=8080 \
todo-backend
```

```
docker run -d --name frontend --network todo-network \
-p 3000:80 \
todo-frontend
```

2.2. Перевірки працездатності

Ключові перевірки: - curl -i http://localhost:3000/api/health
-> HTTP/1.1 200 OK. - curl -i http://localhost:3000/api/todos ->
HTTP/1.1 200 OK. - POST /api/todos через frontend/nginx успішний
(201 Created).

Перевірка резолву імені контейнера БД з backend:

```
docker exec backend getent hosts db
```

Отримана адреса: - 172.18.0.2 db

2.3. Відповіді на запитання

1. Який порядок запуску є практично доцільним і чому?
 - db -> backend -> frontend, бо backend залежить від доступності БД, а frontend залежить від backend.
2. Які команди використовували для запуску всіх контейнерів?
 - Наведені вище у розділі 2.1 (повні команди з аргументами).
3. Яку адресу отримано при резолві імені контейнера БД з backend?

- 172.18.0.2 (у цьому запуску).
4. Що змінилось порівняно з ручним запуском у лабораторній 1?
- Компоненти ізольовані в контейнерах.
 - Залежності не встановлюються локально на хості для запуску застосунку.
 - Мережева взаємодія йде через Docker DNS (імена контейнерів), а не через localhost між процесами хоста.
5. Навіщо nginx у контейнері фронтенду проксює /api/* на бекенд?
- Єдина точка входу для браузера (frontend + API через один origin), простіше конфігурувати клієнт.
 - Уникає проблем CORS у типовому сценарії.
 - Дозволяє звертатись до backend за внутрішнім DNS-іменем Docker (backend).

Завдання 3. Docker Compose (додаткове завдання)

3.1. Опис застосунку

Створено docker-compose.yml з трьома сервісами: - db (postgres:16, volume, env) - backend (build з ./backend, env, depends_on від здорової БД) - frontend (build з ./frontend, порт 3000:80)

3.2. Запуск однією командою

```
docker compose up --build
```

У перевірці використовувався фоновий режим:

```
docker compose up --build -d
```

3.3. Відповіді на запитання

1. Що відбувається при docker compose down, docker compose up -d? Чи зберігаються дані? Чому?
 - Контейнери та мережа видаляються, але volume лишається.
 - Після up -d дані з БД зберігаються.
 - Фактично перевірено: створений запис compose-persist-marker залишився після down/up -d.
2. Що відбувається при docker compose down -v, docker compose up -d? Чи зберігаються дані? Чому?
 - down -v видаляє також volume.

- Після `up -d` БД ініціалізується заново, дані не зберігаються.
- Фактично перевірено: після `down -v/up -d` список задач порожній (`[]`).

3.4. Посилання на створений файл у репозиторії

- `docker-compose.yml`